

OSSP

1. Processid

```
#include <stdio.h>
#include <unistd.h>
int main()
{
printf("My Process ID (PID) = %d\n", getpid());
return 0;
}
```

Output: -

```
shashank@LAPTOP-24KPROR8:~$ nano processid.c
shashank@LAPTOP-24KPROR8:~$ gcc processid.c -o processid
shashank@LAPTOP-24KPROR8:~$ ./processid
My Process ID (PID) = 606
```

2. Process Child

```
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid=fork();

if(pid==0)
{
printf("Child Process\n");
printf("PID = %d\n", getpid());
}
else{
printf("Parent Process\n");
printf("PID = %d\n", getpid());
}
return 0;
}
```

Output:-

```
shashank@LAPTOP-24KPROR8:~$ nano processchild.c
shashank@LAPTOP-24KPROR8:~$ gcc processchild.c -o processchild
shashank@LAPTOP-24KPROR8:~$ ./processchild
Parent Process
PID = 801
Child Process
PID = 802
```

3. Parent waits for child (wait())

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid=fork();
    if(pid==0)
    {
        printf("Child Process Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Process Continues\n");
    }
    return 0;
}
```

Output: -

```
shashank@LAPTOP-24KPROR8:~$ nano processchild.c
shashank@LAPTOP-24KPROR8:~$ nano ParentWchild.c
shashank@LAPTOP-24KPROR8:~$ gcc ParentWchild.c -o ParentWchild
shashank@LAPTOP-24KPROR8:~$ ./ParentWchild
Child Process Executing...
Child Finished
Parent Process Continues
```

4. Multiple Child Process

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    for(int i=1;i<=3;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID = %d\n", i, getpid());
            return 0;
        }
    }
    return 0;
}
```

Output: -

```
shashank@LAPTOP-24KPR0R8:~$ nano Multichild.c
shashank@LAPTOP-24KPR0R8:~$ gcc Multichild.c -o Multichild
shashank@LAPTOP-24KPR0R8:~$ ./Multichild
Child 1 PID = 1166
Child 2 PID = 1167
Child 3 PID = 1168
```

5. Execute Linux command using execlp()

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    printf("Before exec()\n");
    execlp("ls", "ls", "-l", NULL);
    printf("This will not execute.\n");
    return 0;
}
```

Output: -

```
shashank@LAPTOP-24KPROR8:~$ nano Exclp.c
shashank@LAPTOP-24KPROR8:~$ gcc Exclp.c -o Exclp
shashank@LAPTOP-24KPROR8:~$ ./Exclp
Before exec()
total 360
-rwxr-xr-x 1 shashank shashank 16000 Aug  3 13:46 Display
-rw-r--r-- 1 shashank shashank  110 Jul 31 09:56 Display.c
-rwxr-xr-x 1 shashank shashank 15992 Aug 14 10:09 Exclp
-rw-r--r-- 1 shashank shashank  157 Aug 14 10:09 Exclp.c
-rwxr-xr-x 1 shashank shashank 16080 Aug  3 14:01 Exec
-rw-r--r-- 1 shashank shashank  264 Aug  3 14:00 Exec.c
-rwxr-xr-x 1 shashank shashank 15992 Aug  3 13:47 Fork
-rw-r--r-- 1 shashank shashank  104 Jul 31 10:00 Fork.c
-rwxr-xr-x 1 shashank shashank 16040 Aug 14 10:04 Multichild
-rw-r--r-- 1 shashank shashank  162 Aug 14 10:04 Multichild.c
-rw-r--r-- 1 shashank shashank    0 Aug  4 13:22 OS.txt
drwxr-xr-x 2 shashank shashank  4096 Aug  4 13:34 OSLab
-rwxr-xr-x 1 shashank shashank 16080 Aug 14 09:56 ParentWchild
-rw-r--r-- 1 shashank shashank  257 Aug 14 09:57 ParentWchild.c
-rwxr-xr-x 1 shashank shashank 16000 Aug  3 13:50 Parentchild
-rw-r--r-- 1 shashank shashank  230 Aug  3 13:50 Parentchild.c
-rwxr-xr-x 1 shashank shashank 16304 Aug  7 09:27 Prac1
-rw-r--r-- 1 shashank shashank  438 Aug  7 09:26 Prac1.c
-rwxr-xr-x 1 shashank shashank 16168 Aug 10 11:02 Prac2
-rw-r--r-- 1 shashank shashank  443 Aug 10 11:04 Prac2.c
-rwxr-xr-x 1 shashank shashank 16000 Aug 11 04:07 Processid
-rw-r--r-- 1 shashank shashank  118 Aug 11 04:07 Processid.c
-rwxr-xr-x 1 shashank shashank 16056 Aug  3 13:44 User
-rw-r--r-- 1 shashank shashank  158 Jul 31 09:55 User.c
-rwxr-xr-x 1 shashank shashank 16032 Aug  3 13:55 Wait
-rw-r--r-- 1 shashank shashank  274 Aug  3 13:55 Wait.c
-rwxr-xr-x 1 shashank shashank 15992 Aug  3 13:57 Wait2
-rw-r--r-- 1 shashank shashank  159 Aug  3 13:57 Wait2.c
-rw-r--r-- 1 shashank shashank  155 Aug  4 13:50 firstpg.txt
-rwxr-xr-x 1 shashank shashank 15952 Aug  4 14:18 hello
-rw-r--r-- 1 shashank shashank   72 Aug  4 14:00 hello.c
-rw-r--r-- 1 shashank shashank   69 Aug 10 11:02 output.txt
-rwxr-xr-x 1 shashank shashank 16312 Aug  9 13:23 practical1
-rw-r--r-- 1 shashank shashank  443 Aug  9 13:28 practical1.c
-rwxr-xr-x 1 shashank shashank 16088 Aug 14 09:46 processchild
-rw-r--r-- 1 shashank shashank  225 Aug 14 09:47 processchild.c
-rwxr-xr-x 1 shashank shashank 16000 Aug 14 09:37 processid
-rw-r--r-- 1 shashank shashank  112 Aug 14 09:39 processid.c
-rw-r--r-- 1 shashank shashank   75 Jul 31 09:43 samplePg.c
-rw-r--r-- 1 shashank shashank   69 Aug 10 10:59 somefile.txt
```

6. Parent and child Executing concurrently

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();
    for(int i=1;i<=5;i++)
    {
        printf("PID: %d Count: %d\n", getpid(), i);
        sleep(1);
    }
    return 0;
}
```

Output : -

```
shashank@LAPTOP-24KPR0R8:~$ nano Exclp.c
shashank@LAPTOP-24KPR0R8:~$ nano PC.c
shashank@LAPTOP-24KPR0R8:~$ gcc PC.c -o PC
shashank@LAPTOP-24KPR0R8:~$ ./PC
PID: 1265 Count: 1
PID: 1266 Count: 1
PID: 1265 Count: 2
PID: 1266 Count: 2
PID: 1265 Count: 3
PID: 1266 Count: 3
PID: 1265 Count: 4
PID: 1266 Count: 4
PID: 1265 Count: 5
PID: 1266 Count: 5
```